Project Brief: Change-Aware, Feasibility-First Scheduling for AI Inference

AMS 560 Big Data Systems, Algorithms and Networks • Stony Brook University • Fall 2026 • Group 20

Team: Krupesh, Parikshit, Jyotir, Sur, Riya • Instructor: Prof. Zhenhua Liu • Brief written Oct 21, 2026

Note to the AI assistant reading this

This document is the full context for the project. You have no other history. Please:

- Treat Sections 1 to 12 as decided scope. Do not change the project into something else (for example, do not build a serving engine).
- Your first job: create the project folder structure in **Section 13**, with a README in each folder saying what goes there and who owns it, empty Python modules, and the three contract docs filled from **Section 10**.
- Values marked **(default)** are starting choices the team can change. Values marked **(frozen)** must not change without the team agreeing.
- Explain things in plain, simple words. The team is learning this area while building it.
- Open questions are listed in **Section 15**. Ask before deciding them.

1. The project in plain words

A company runs the same AI model (an LLM) at **5 to 10 places**. A place is a data center or cloud region where the company rents GPUs and runs a copy of the model. We call each place a **node**. Every user request must be sent to one node. Some nodes are close, some are cheap, some are busy, some are hot or short on power, and these things **change over time** (a traffic spike, a price change, a node overheats, the network slows, a node goes down).

We build the **scheduler (router)** that picks the node for each request. It works in a loop of four steps:

Step	What happens
1. Filter	Remove nodes that cannot or must not take the request: model not loaded, not enough GPU memory, not enough power headroom, too hot, or down. If no node is left, reject the request on purpose and record why.
2. Choose	Among the remaining nodes, pick one with **sliding-window LinUCB**, a learning method that uses information about the request and the nodes.
3. Send	Send the request to that node (in our case, a simulated node).
4. Learn	See what happened (on time? how slow? how costly?) and update the policy. Keep only **recent** results, so old information about a node that has since changed is forgotten.

The router is real code (an HTTP service). The nodes are **simulated** in Python. We test the router under six controlled change events and measure how well it serves requests and **how fast it recovers** after each change.

What this project is NOT

- Not a serving engine. We do not build or change vLLM / SGLang internals (batching, KV cache management inside one GPU). That is a different layer.

- Not a real multi-data-center deployment. Nodes are simulated. Calibrating one simulated node against a real vLLM run is optional.
- Not a new bandit algorithm. LinUCB and sliding-window LinUCB already exist. Our contribution is the full system and the recovery evaluation (see Section 3).

Router vs serving-engine scheduler (common confusion)

	Our router (this project)	Serving-engine scheduler (not this project)
Question it answers	Which node (place) gets this request?	Inside one node, which requests run together on the GPU right now?
Example tools	Load balancers, geo routers, our code	vLLM, SGLang, TensorRT-LLM
Sees	Summary numbers from every node	Every request and memory block inside one GPU server
Time scale	Once per request	Every few milliseconds (each model step)

2. Words you need

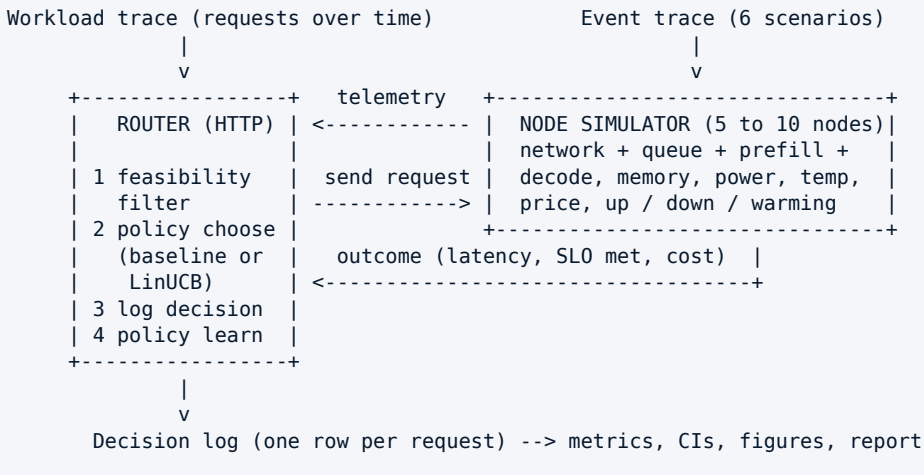
Term	Plain meaning
Node / site	One place (data center or cloud region) with rented GPUs running a copy of the model.
Request	One user prompt sent to the model. Has an input length and an output length, counted in tokens.
Token	A piece of a word. Models read and write text token by token.
Prefill	The first phase: the model reads the whole prompt at once. Time grows with input length. Limited by GPU compute.
Decode	The second phase: the model writes the answer one token at a time. Time grows with output length. Limited by GPU memory speed, and slower when the node is busier.
TTFT	Time to first token: how long until the user sees the first word. = network + waiting in queue + prefill.
KV cache	Memory each running request uses on the GPU. When it is full, a node cannot take more requests.
SLO	Service level objective: the promise, e.g. TTFT under 250 ms.
Goodput	Requests per second that finish **within** their SLO. Late requests do not count. Main quality number.
p95 / p99 latency	The time that 95% / 99% of requests beat. Shows the slow tail, not just the average.
Telemetry	The numbers each node reports over time (queue length, memory used, temperature, price, ...).
Feasibility filter	Hard rules that remove unsafe or impossible nodes before any choice is made.
Contextual bandit	A learning method that picks one option per round using context, sees a reward only for the option it picked, and improves over time.
LinUCB	A contextual bandit: predicts each option's reward with a linear model and adds a bonus for options it is unsure about, so it keeps exploring.
Sliding window	Learn only from the last W results, so the policy forgets stale history and adapts after changes.
Adaptation time	How long after a change until goodput is back near its new normal level. Exact definition in Section 8.
Baseline	A simple policy we compare against (round-robin, closest node, ...).
Ablation	Turn off one part (filter, context, forgetting) to show how much that part helps.

3. Professor's feedback (must be addressed)

Comment	How we address it
"Frame the novelty around the full scheduling system and recovery evaluation, not around sliding-window LinUCB itself."	Sliding-window LinUCB is cited as existing work (Cheung, Simchi-Levi, Zhu 2019). Our claim is: (1) an integrated loop of feasibility filter + context + forgetting for multi-site inference, and (2) a recovery benchmark with six controlled change events and a precise adaptation-time metric.
"Define 'adaptation time' precisely, e.g. time needed to return within a fixed percentage of the new steady-state goodput after a change."	Exact definition in Section 8 (frozen once the team signs off). Reported for every change event, with deepest dip, lost goodput, and the new steady level.

Novelty sentence for the report: "We contribute an integrated, feasibility-first, change-aware scheduling loop for multi-site LLM inference and a recovery benchmark with six controlled change events. Sliding-window LinUCB is an existing method that we use inside this loop."

4. System design



Component	Job	Owner
Router service	HTTP endpoint; runs filter, policy, logging for each request	Krupesh
Node simulator	Simulates each node's state and the time each request takes	Parikshit
Telemetry	Versioned schema of what nodes report; collection over time	Parikshit
Event traces	Replayable files for the six change scenarios	Parikshit
Feasibility filter	Hard rules, removal reasons, controlled rejection	Jyotir
Baseline policies	Round-robin, min RTT, least load, lowest energy, static weighted, (power of two)	Jyotir
Bandit policies	LinUCB and sliding-window LinUCB; context vector; reward	Sur
Decision log	One row per request with choice, reasons and outcome	Krupesh
Evaluation	Metrics, adaptation time, bootstrap CIs, figures, experiment runner	Riya

All policies share one interface and all pass through the same filter, so comparisons are fair: only the choosing step differs.

5. Node simulator

Each node keeps this state over time: network delay to each user region, queue of waiting requests, running requests, GPU memory / KV cache used, models loaded, power headroom, electricity price, GPU rent price, temperature, status (up, down, warming up).

Time for one request on node *n*, built from four parts:

$$T_{total} = T_{network} + T_{queue} + T_{prefill} + T_{decode}$$

$$TTFT = T_{network} + T_{queue} + T_{prefill}$$

Simple models for the two GPU phases (default; constants fitted per node type):

$T_{prefill} = a_p + b_p \cdot (\text{input tokens})$ — a fixed start cost plus a cost per prompt token

$T_{decode} = (\text{output tokens}) \cdot t_{tok} \cdot (1 + c \cdot load)$ — time per output token, slower when the node is busier

Here *load* is running requests divided by the node's capacity, and *c* sets how much a busy node slows down. A node cannot accept a request if its KV cache memory would overflow.

Data sources (use real data where possible)

What	Source (default)
Request arrival times and token lengths	Azure LLM inference traces (public)
Electricity price over time	NYISO / CAISO public hourly prices
Network delay between cities	Public inter-city ping data (e.g. WonderNetwork)
Prefill / decode speed per GPU	Published benchmarks, or optional calibration against one real vLLM run
Simulation library	SimPy (discrete-event simulation in Python)

Six scenarios (frozen list; timing values are defaults)

Each run is 600 s of simulated time. The change starts at t_0 = 200 s, so there is a steady period before and enough time after to measure recovery.

#	Scenario	What changes at t_0
S1	Stable	Nothing. Checks normal behavior and the cost of exploring.
S2	Traffic burst	Arrival rate jumps (e.g. 3×) for 120 s.
S3	Price / renewable shift	Electricity price at the cheapest nodes jumps; another region becomes cheap.
S4	Thermal throttle	One busy node gets hot, slows down, and can cross the temperature limit.
S5	Network degradation	Delay to one popular node rises sharply (e.g. +80 ms).
S6	Outage + recovery	One node goes down at t_0; returns at 400 s but needs a warm-up period before it is fully fast.

6. The scheduling policy

6.1 Feasibility filter (hard rules)

A node is allowed for a request only if **all** are true. Limits live in a config file, not in code.

Rule	Check
Model available	The requested model is loaded on the node
Memory	KV cache has room for this request (input + expected output tokens)
Power	Power headroom is above the safety limit
Temperature	Temperature is below the safety limit
Status	Node is up (a warming node may be allowed with a penalty; default: allowed)

Every removed node gets a reason in the log. If no node is left, the request is **rejected on purpose** with reason "no feasible node". The policy never overrides the filter to save cost.

6.2 Context vector

For each candidate node, the policy sees a short list of numbers (scaled to similar ranges): request input length, expected output length, request class / SLO, network delay to that node, queue length, running requests, KV cache used, electricity and GPU price, temperature, and a warming-up flag. Engine signals (queue, running requests, KV cache) are expected to matter most.

6.3 Reward (after each request)

$$r = w_1 \cdot [\text{SLO met}] - w_2 \cdot \text{latency} - w_3 \cdot \text{energy cost} - w_4 \cdot \text{thermal stress}$$

[SLO met] is 1 if the request met its SLO and 0 if not. Each other term is scaled to 0..1 first. The weights w are in config (default: SLO term dominates).

6.4 LinUCB (Li et al., 2010)

For each node *a*, keep a matrix A_a and a vector b_a. Start with $A_a = \lambda I$ ($\lambda = 1$) and $b_a = 0$.

Step 1, estimate: the learned weights for node *a* are

$$\theta_a = A_a^{-1} b_a$$

Step 2, score: for context **x**, the score is the predicted reward plus an uncertainty bonus. α controls how much to explore.

$$\text{score}_a = \theta_a^{T} \mathbf{x} + \alpha \cdot \sqrt{\mathbf{x}^{T} A_a^{-1} \mathbf{x}}$$

Step 3, choose: pick the feasible node with the highest score. **Step 4, update** only the chosen node with reward r:

$$A_a \leftarrow A_a + \mathbf{x}\mathbf{x}^{T}, \quad b_a \leftarrow b_a + r \cdot \mathbf{x}$$

6.5 Sliding-window LinUCB (Cheung, Simchi-Levi, Zhu, 2019)

Same as above, but A_a and b_a are built only from the last **W** decisions. When a result leaves the window, subtract it back out ($A_a \leftarrow A_a - \mathbf{x}\mathbf{x}^{T}$, $b_a \leftarrow b_a - r\,\mathbf{x}$). Small W reacts fast but is noisy; large W is stable but slow. W is swept in experiments (default sweep: 200, 500, 1000, 2000, 5000).

7. Baselines and ablations

Policy	Rule (all run after the same filter)
Round-robin	Take feasible nodes in turn
Min RTT	Closest node (lowest network delay)
Least load	Node with the fewest waiting + running requests
Lowest energy	Cheapest node for energy
Static weighted	Fixed hand-set formula mixing delay, load and price
Power of two (optional)	Pick 2 random feasible nodes, send to the less busy one
Plain LinUCB	LinUCB without the window (never forgets)
Oracle (reference)	Knows true node state; gives the best-possible level for lost-goodput calculation

Ablations of our policy: (a) no feasibility filter, (b) no context (plain bandit), (c) no forgetting (= plain LinUCB). Plus a window-size sweep and an α (exploration) check.

8. How we measure

Metric	Meaning
SLO goodput	Requests per second finishing within SLO (main metric)
p95 / p99 latency	Tail TTFT and total time
Rejection rate	Share of requests rejected because no node was feasible
Cost / energy per successful request	Total energy and GPU cost divided by requests that met SLO
Adaptation time	Defined below
Deepest dip, lost goodput	How bad the drop was, and total goodput lost vs the oracle during recovery

Adaptation time (frozen after team sign-off)

1. A change starts at time t_0.
2. Measure goodput G(t) in 1-second bins, smoothed with a 5-second moving average.
3. The new steady level G* is the average of G(t) from t_0 + 60 s to t_0 + 120 s.
4. Adaptation time = (first time t after t_0 where $G(t) \geq 0.9 \cdot G^*$ and it stays $\geq 0.9 \cdot G^*$ for 10 s in a row) − t_0.
5. If that never happens, report "did not recover" instead of a number.
6. Always report G* next to adaptation time. A weak policy can settle at a low level and look fast; showing G* (and the oracle's level) exposes that.

Experiment rules

- Every policy × every scenario × at least **10 random seeds**. Same seeds and traces for every policy.
- **95% bootstrap confidence intervals** on every reported number.
- Main figure per scenario: goodput over time around t_0, one line per policy, with the adaptation-time marker.
- Everything reproducible from config files + seeds with one command.

Success criteria

- **Better:** higher SLO goodput or lower cost than the strongest baseline.
- **Faster:** shorter adaptation time than plain LinUCB after changes.
- **Safe:** never breaks a hard rule to save cost.
- A clean negative result is acceptable if it explains when the method helps and when it does not.

9. Team and ownership

Person	Owns	Main deliverables	Hands to
Krupesh	Architecture + integration	API contract, router service, decision log format, weekly integrated build, novelty wording	Running system + logs to Riya
Parikshit	Simulator + telemetry	Simulator (5 to 10 nodes) + tests, telemetry schema, six event traces, real-data loaders	Telemetry to router/filter/policy; traces to Riya
Jyotir	Feasibility + static baselines	Filter module + tests, 5 (or 6) baselines, baseline log on all traces	Filter to Sur/router; baseline log to Riya
Sur	LinUCB + sliding window	Plain and windowed LinUCB, context vector, reward, window sweep, citations	Policy to router; sweep to Riya
Riya	Experiments + analysis + report	Metrics incl. adaptation time, experiment runner, CIs, ablations, figures, report	Report + figures to all
All	Validation + recording	End-to-end test, final review, Zoom recording	Submission package

10. Interface contracts (frozen at end of Week 1)

These let everyone work in parallel. Any change needs a message to the whole group. Field lists are the starting version (v1).

10.1 Telemetry record (one per node per tick) — owner Parikshit

```
{
  "schema_version": "1.0",
  "t": 312.0,                       // simulation time, seconds
  "node_id": "us-east-1",
  "status": "up",                   // up | down | warming
  "rtt_ms": {"nyc": 12, "chi": 25, "sfo": 70},
  "queue_len": 7,
  "running": 22,
  "capacity": 32,                   // max running requests
  "kv_used_frac": 0.68,
  "models_loaded": ["llama-3-8b"],
  "power_headroom_kw": 4.5,
  "temperature_c": 71.0,
  "energy_price_usd_kwh": 0.11,
  "gpu_price_usd_hr": 2.40
}
```

10.2 Routing API — owner Krupesh

```
POST /route
request:  {"request_id": "r-000123", "t_arrival": 312.4, "user_region": "nyc",
           "model": "llama-3-8b", "input_tokens": 850, "expected_output_tokens": 200,
           "slo_class": "interactive", "ttft_slo_ms": 250}
response: {"request_id": "r-000123", "decision": "route",      // route | reject
           "node_id": "us-east-1", "policy": "sw_linucb",
           "feasible_nodes": ["us-east-1", "us-central-1"],
           "filtered_out": {"us-west-2": "temperature", "eu-1": "model_missing"},
           "reject_reason": null}

POST /feedback   (simulator -> router, after the request finishes)
          {"request_id": "r-000123", "ttft_ms": 184, "e2e_ms": 2210, "slo_met": true,
           "energy_kwh": 0.0004, "cost_usd": 0.00013}
```

Policy interface (Python):

```
class Policy:
    def choose(self, request, candidates: list[NodeState]) -> str: ...   # returns node_id
    def update(self, request, node_id: str, outcome: Outcome) -> None: ...
```

10.3 Decision log (one row per request, CSV or Parquet) — owner Krupesh, used by Riya

run_id, seed, scenario, policy, request_id, t_arrival, decision, node_id, feasible_nodes, filtered_out, reject_reason, ttft_ms, e2e_ms, slo_met, energy_kwh, cost_usd, reward

10.4 Experiment bundle — owner Riya

For each run: the exact config used, seed, git commit, decision log, summary metrics table. Figures are regenerated from bundles only.

11. Timeline

Four weeks, core features frozen at the end of Week 3. Calendar dates are not set yet (see Section 15).

Week	Goal	Done when
1. Foundation	Contracts, simulator skeleton, telemetry + tests	4 contracts frozen; 5 nodes produce telemetry; router skeleton answers /route
2. Baseline system	HTTP router + filter + static baselines end to end	All baselines run on S1 through the filter; decision log written; metrics compute
3. Adaptive system	LinUCB, sliding window, all six event traces	SW-LinUCB runs on all scenarios; first full results; **core freeze**
4. Buffer + finalize	10-seed runs, CIs, ablations, report, video	Report, figures, repo and Zoom recording submitted

12. Tech stack (default)

Need	Tool
Language	Python 3.11+
Simulation	SimPy
Router service	FastAPI + Uvicorn (an in-process mode without HTTP is allowed for fast experiment runs)
Math / data	NumPy, pandas, PyArrow (Parquet logs)
Configs	YAML files, loaded with Pydantic models for checking
Plots	Matplotlib
Tests	pytest
Code style	ruff (lint + format)
Version control	Git; one shared repo; small pull requests reviewed by one teammate

13. Project folder structure (build this first)

Repository name: **geo-inference-scheduler**. Python package name: **geosched**. Each folder gets a short README.md saying what goes there and who owns it.

```
geo-inference-scheduler/
├── README.md                    # what the project is, how to install, how to run one experiment
├── pyproject.toml               # dependencies, ruff + pytest settings
├── Makefile                     # make install | test | run-one | run-all | figures
├── .gitignore                   # ignore data/raw, results/runs, caches
│
├── docs/                                    # owner: all (Krupesh keeps it tidy)
│   ├── project_brief.pdf        # this document
│   ├── api_contract.md          # Section 10.2
│   ├── telemetry_schema.md      # Section 10.1
│   ├── decision_log.md          # Section 10.3
│   ├── metrics.md               # Section 8, incl. adaptation-time definition
│   └── decisions.md             # dated list of team decisions
│
├── configs/
│   ├── nodes/                   # owner: Parikshit
│   │   ├── nodes_5.yaml         # 5-node setup (location, capacity, GPU type, limits)
│   │   └── nodes_10.yaml
│   ├── scenarios/               # owner: Parikshit
│   │   ├── s1_stable.yaml
│   │   ├── s2_traffic_burst.yaml
│   │   ├── s3_price_shift.yaml
│   │   ├── s4_thermal_throttle.yaml
│   │   ├── s5_network_degradation.yaml
│   │   └── s6_outage_recovery.yaml
│   ├── feasibility.yaml         # owner: Jyotir (limits for the hard rules)
│   ├── policies/                # owner: Sur (bandits), Jyotir (baselines)
│   │   ├── baselines.yaml
│   │   ├── linucb.yaml
│   │   └── sw_linucb.yaml       # alpha, lambda, window W, reward weights
│   └── experiments/             # owner: Riya
│       ├── smoke_test.yaml      # 1 policy, 1 scenario, 1 seed (fast check)
│       ├── main_grid.yaml       # all policies x 6 scenarios x 10 seeds
│       ├── window_sweep.yaml
│       └── ablations.yaml
│
├── data/                                    # owner: Parikshit
│   ├── raw/                     # downloaded traces (not in git)
│   └── processed/               # cleaned traces ready to replay
│
├── src/geosched/
│   ├── __init__.py
│   ├── common/                  # owner: Krupesh — shared types used by everyone
│   │   ├── types.py             # Request, NodeState, Decision, Outcome
│   │   └── config.py            # load + check YAML configs
│   ├── simulator/               # owner: Parikshit
│   │   ├── node.py              # one node: state, queue, capacity, memory
│   │   ├── latency_model.py     # network + queue + prefill + decode
│   │   ├── cluster.py           # all nodes, simulation clock (SimPy)
│   │   ├── workload.py          # request arrivals from traces
│   │   └── events.py            # applies scenario changes over time
│   ├── telemetry/               # owner: Parikshit
│   │   ├── schema.py            # telemetry record v1
│   │   └── collector.py         # snapshots node state for the router
│   ├── feasibility/             # owner: Jyotir
│   │   └── filter.py            # hard rules -> allowed nodes + reasons
│   ├── policies/
│   │   ├── base.py              # owner: Krupesh — Policy interface
│   │   ├── registry.py          # name -> policy class, for configs
│   │   ├── baselines/           # owner: Jyotir
```

```
│   │   │   ├── round_robin.py
│   │   │   ├── min_rtt.py
│   │   │   ├── least_load.py
│   │   │   ├── lowest_energy.py
│   │   │   ├── static_weighted.py
│   │   │   └── power_of_two.py
│   │   ├── bandits/             # owner: Sur
│   │   │   ├── features.py      # builds the context vector
│   │   │   ├── linucb.py
│   │   │   └── sw_linucb.py
│   │   └── oracle.py            # owner: Riya (reference only)
│   ├── reward/                  # owner: Sur
│   │   └── reward.py
│   ├── router/                  # owner: Krupesh
│   │   ├── app.py               # FastAPI: /route, /feedback, /health
│   │   └── engine.py            # filter -> policy -> log (also used without HTTP)
│   ├── decision_log/            # owner: Krupesh
│   │   └── writer.py
│   └── evaluation/              # owner: Riya
│       ├── metrics.py           # goodput, p95/p99, rejection, cost per success
│       ├── adaptation.py        # adaptation time, deepest dip, lost goodput
│       ├── bootstrap.py         # 95% confidence intervals
│       └── plots.py
│
├── experiments/                                # owner: Riya
│   ├── run_experiment.py        # runs one config -> results/runs/<run_id>/
│   └── run_grid.py              # runs a whole grid (policies x scenarios x seeds)
│
├── scripts/                                    # owner: Parikshit
│   ├── download_data.py
│   └── preprocess_traces.py
│
├── tests/
│   ├── unit/                    # each owner tests their own module
│   │   ├── test_latency_model.py
│   │   ├── test_filter.py
│   │   ├── test_baselines.py
│   │   ├── test_linucb.py
│   │   └── test_adaptation.py
│   └── integration/             # owner: Krupesh
│       └── test_end_to_end.py   # smoke_test.yaml runs and writes a decision log
│
├── notebooks/                   # free exploration, not used for final numbers
├── results/
│   ├── runs/                    # one folder per run (not in git)
│   ├── tables/                  # final summary tables
│   └── figures/                 # final figures
└── report/                                     # owner: Riya (sections from all)
    ├── report.md                # or LaTeX
    ├── figures/
    └── slides/
```

Rules for the repo

- Code goes only in **src/geosched**. Numbers in the report come only from **results/**, which come only from **configs/** + seeds.
- No limits, weights or window sizes hard-coded in Python: put them in **configs/**.
- Every module has at least one unit test before it is merged.
- Each folder's README.md names its owner and what goes in it.

14. Background reading (for learning)

Source	Why it matters
Philip Kiely, *Inference Engineering* (book) — ch. 2.2, 2.4, 7.2, 7.3	How serving works: prefill vs decode, batching, KV cache, routing and load balancing across replicas and regions
Li et al., 2010, "A Contextual-Bandit Approach to Personalized News Article Recommendation"	Original LinUCB
Garivier & Moulines, 2011, "On Upper-Confidence Bound Policies for Switching Bandit Problems"	Why forgetting old data helps when things change
Cheung, Simchi-Levi, Zhu, 2019, "Learning to Optimize under Non-Stationarity"	Sliding-window LinUCB (the method we use)
Zhong et al., 2024, "DistServe" (OSDI)	Goodput as the main serving metric; prefill vs decode
Dean & Barroso, 2013, "The Tail at Scale"	Why p95 / p99 matter
Mitzenmacher, 2001, "The Power of Two Choices in Randomized Load Balancing"	A strong simple baseline

15. Open questions (ask the team before deciding)

- Calendar dates for Weeks 1 to 4.
- 5 or 10 nodes in the main experiments (or both).
- Window counted over all decisions (as in Cheung et al.) or per node.
- Final reward weights and the exact feasibility limits.
- Whether to calibrate one simulated node against a real vLLM run (optional extra).
- Final values for the adaptation-time settings (90%, 10 s hold, 60 to 120 s steady window) before they are frozen.
- Report format: Markdown or LaTeX.

Suggested first message to a new chat: "Read this project brief. Then create the folder structure in Section 13 as a Git repo: README files per folder with owner and purpose, empty Python modules with docstrings, pyproject.toml, Makefile, .gitignore, and docs/ filled from Sections 8 and 10. Keep explanations simple."